Labo 12 — Labo pratique : déployer, mettre à jour, revenir en arrière

Objectif : déployer l'API en deux réplicas avec des sondes, la mettre à l'échelle, la passer en version 1.1 sans coupure, tenter une version cassée et observer le rollout se bloquer, revenir en arrière, puis casser volontairement une readiness et une liveness pour voir leurs effets opposés.

Prérequis — Labo 11 terminé : k3s installé, `localhost/api:1.0` importée dans le cluster, Podman avec `api:1.0`.

Fichiers fournis (`files/`) - `deployment-api.yaml` — le Deployment de l'API : deux réplicas, readiness et liveness sur `/actuator/health`. - `Dockerfile.v11` — construit `api:1.1` (un autre message par défaut). - `Dockerfile.bad` — construit `api:bad` (une image qui ne démarre pas).

Étape 0 — Deux images de plus dans le cluster

```
mkdir -p ~/labo-docker/12 && cd ~/labo-docker/12 && cp <chemin-du-labo>/files/* .
podman build -q -f Dockerfile.v11 -t api:1.1 . && podman build -q -f Dockerfile.bad -t api:bad
.
podman save localhost/api:1.1 | sudo k3s ctr images import -
podman save localhost/api:bad | sudo k3s ctr images import -
sudo k3s crictl images | grep 'localhost/api'
```

Observez deux builds en cache (quelques secondes), deux imports, et trois lignes `localhost/api` dans le cluster : `1.0`, `1.1`, `bad`.

Étape 1 — Le Deployment

```
kubectl create namespace lab12 ; kubectl config set-context --current --namespace=lab12
kubectl apply -f deployment-api.yaml
kubectl rollout status deployment/api --timeout=60s
kubectl get deployment,replicaset,pods -o wide
```

Observez `Waiting for deployment "api" rollout to finish: 1 of 2 updated replicas are available...` puis `successfully rolled out` ; trois niveaux d'objets : `deployment.apps/api 2/2`, `replicaset.apps/api-6c7bb54fc5 2 2 2`, et deux Pods `api-6c7bb54fc5-xxxxx Running` avec chacun leur IP.

```
kubectl get pods --show-labels
kubectl get pods -l app=api -o name
kubectl describe deployment api | grep -E 'Replicas:|StrategyType|RollingUpdateStrategy'
```

Observez les labels `app=api,pod-template-hash=6c7bb54fc5` sur chaque Pod, les deux Pods filtrés par `-l app=api`, et la stratégie : `RollingUpdate`, `25% max unavailable, 25% max surge`.

Explication. Vous avez écrit un Deployment ; le cluster a créé un ReplicaSet (nommé d'après le hash du template) et deux Pods, reliés par les labels. Le `rollout status` attendait que les readiness probes passent.

Étape 2 — Mettre à l'échelle

```
kubectl scale deployment api --replicas=3
kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl get rs
```

Observez un troisième Pod avec le **même** hash, et le ReplicaSet à `3 3 3`.

Explication. `scale` change `spec.replicas` ; le ReplicaSet fait le reste. Aucun nouveau ReplicaSet : le template n'a pas changé.

Étape 3 — Passer en 1.1 sans coupure

```
kubectl set image deployment/api api=localhost/api:1.1
kubectl rollout status deployment/api --timeout=90s
kubectl get rs
kubectl rollout history deployment/api
kubectl exec $(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}') -- wget -
qO- http://localhost:8080/ ; echo
kubectl get events --sort-by=.lastTimestamp | grep ScalingReplicaSet | tail -6 | cut -c1-110
```

Observez le suivi `1 out of 3 new replicas have been updated…`, `2 out of 3…`, `1 old replicas are pending termination…`, `successfully rolled out` ; deux ReplicaSets, le nouveau à `3 3 3` et l'ancien à `0 0 0` ; deux révisions dans l'historique ; `{"message":"version 1.1",…}` ; et la danse des Events : nouveau `0 → 1`, `1 → 2` / ancien `3 → 2`, ancien `2 → 1` / nouveau `2 → 3`, ancien `1 → 0`.

Explication. Jamais moins de trois Pods prêts : un nouveau est créé, attendu prêt, puis un ancien retiré. L'ancien ReplicaSet reste à zéro : c'est la révision 1, prête pour un retour arrière.

Étape 4 — Une version cassée

```
kubectl set image deployment/api api=localhost/api:bad
sleep 25 ; kubectl get pods
kubectl rollout status deployment/api --timeout=20s ; echo "exit=$?"
kubectl get rs
kubectl logs $(kubectl get rs --sort-by=.metadata.creationTimestamp -o name | tail -1) --tail=1
```

Observez trois Pods `1/1 Running` de la 1.1 et **un seul** nouveau Pod `0/1 Error` avec `RESTARTS 2` ; `rollout status` qui attend (`1 out of 3 new replicas have been updated…`) puis `error: timed out waiting for the condition`, `exit=1` ; trois ReplicaSets dont le nouveau à `1 1 0` ; et la cause, lue dans les logs du nouveau ReplicaSet : `demarrage impossible : configuration manquante`.

```
kubectl rollout undo deployment/api
kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl rollout history deployment/api
kubectl get deployment api -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
```

Observez `deployment.apps/api rolled back`, le Pod cassé en `Terminating` (ou encore `Error` le temps qu'il disparaisse), les trois Pods de la 1.1 intacts, un historique `1, 3, 4` (la révision 2 est devenue 4 par le retour arrière), et l'image revenue à `localhost/api:1.1`.

Explication. Le Deployment n'a jamais retiré un seul Pod sain : `maxUnavailable` de 25 % sur trois réplicas vaut zéro. Le service n'a pas été touché pendant l'incident, et le retour arrière a pris une commande.

Étape 5 — Une readiness qui échoue

```
kubectl patch deployment api --type=json -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","valu
e":8081}]'
sleep 25 ; kubectl get pods
kubectl describe pods -l app=api | grep 'Readiness probe failed' | tail -1 | cut -c1-120
kubectl get deployment api
kubectl rollout status deployment/api --timeout=10s ; echo "exit=$?"
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
```

Observez un nouveau Pod `0/1 Running` (il tourne, mais rien n'écoute sur `8081`) à côté des trois anciens `1/1` ; `Readiness probe failed: Get "http://10.42.0.x:8081/actuator/health": dial tcp … connection refused` ; `READY 3/3 UP-TO-DATE 1 AVAILABLE 3` ; un `rollout status` qui expire ; puis le retour en arrière.

Explication. Un Pod jamais prêt ne reçoit pas de trafic et ne permet pas de retirer un ancien : le rollout se bloque sans dommage. C'est le comportement que vous voulez quand une nouvelle version ne démarre pas correctement.

Étape 6 — Une liveness qui échoue

```
kubectl patch deployment api --type=json -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/httpGet/port","value
":8081},
{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/initialDelaySeconds","va
lue":2},
{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/periodSeconds","value":3
}]'
sleep 40 ; kubectl get pods
kubectl describe pods -l app=api | grep -E 'Liveness probe failed|Killing' | tail -2 | cut -c1-
120
```

Observez cette fois **trois** nouveaux Pods, tous `0/1`, en `CrashLoopBackOff` (ou `Running` entre deux relances) avec `RESTARTS 2` : le rollout a remplacé les anciens (la readiness passait), puis la liveness a tué chaque nouveau Pod ; `Liveness probe failed: … connection refused` et `Container api failed liveness probe, will be restarted`.

```
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl rollout history deployment/api
```

Observez les trois Pods sains de retour, les cassés en `Terminating`, et un historique qui a encore sauté des numéros.

Explication. Le rollout ne regarde que la readiness ; une liveness cassée agit après, sur des Pods déjà comptés bons, et emporte tout le service. Readiness : protège le déploiement ; liveness : à régler avec prudence.

Nettoyage

```
kubectl delete namespace lab12
kubectl config set-context --current --namespace=default
sudo k3s crictl rmi localhost/api:bad
podman rmi api:bad
```

Observez le namespace supprimé avec ses objets, et l'image cassée retirée du cluster et de Podman. Gardez `api:1.0` et `api:1.1` des deux côtés : le labo 13 déploie la chaîne complète.

Ce que vous devez pouvoir affirmer maintenant

- Un `apply` de Deployment crée un ReplicaSet `api-<hash>` et ses Pods, reliés par `app=api` et `pod-template-hash`.
- `scale` garde le ReplicaSet ; `set image` en crée un nouveau et déplace les réplicas un par un : `0 → 1`, `3 → 2`, … jamais moins de trois prêts.
- Une image cassée = un seul Pod `Error`, trois anciens intacts, `rollout status` en `timed out`, `logs rs/…` pour la cause, `rollout undo` pour réparer.
- Readiness cassée = `0/1 Running`, rollout bloqué, service intact ; liveness cassée = tout en `CrashLoopBackOff`.
- `rollout history` saute des numéros après chaque `undo` : normal.